Marisa

Static dictionaries with fast prefix lookup.

Install & load

```
INSTALL marisa FROM community;
LOAD marisa;
```

01 At a glance

Build a compact MARISA trie from a column of strings. Store the trie as a BLOB and use it for exact membership, autocomplete, or prefix-rule matching.

String column → Trie BLOB → Lookup / prefix

02 Core SQL surface

`marisa_trie(text)`

Aggregate a column of strings into one trie BLOB.

`marisa_lookup(trie, text)`

Test exact membership; returns BOOLEAN.

`marisa_predictive(trie, prefix, limit)`

Return stored strings that begin with the supplied prefix.

`marisa_common_prefix(trie, text, limit)`

Return stored strings that are prefixes of the supplied text.

03 Keep in mind

Two directions of prefix search

Predictive search expands a short input into longer dictionary entries. Common-prefix search finds shorter stored rules matching the start of a longer input.

04 Build a dictionary

```
CREATE TABLE dictionary AS
SELECT marisa_trie(name) AS trie
FROM (VALUES ('Alice'), ('Megan'),
             ('Melissa')) AS t(name);
```

One result row stores the entire dictionary.

05 Lookup & autocomplete

```
SELECT marisa_lookup(trie, 'Alice') AS found,
       marisa_predictive(trie, 'Me', 10)
         AS suggestions
FROM dictionary;
```

found is true; suggestions includes Megan and Melissa.

06 Match stored prefixes

```
WITH rules AS (
  SELECT marisa_trie(s) AS trie
  FROM (VALUES ('U'), ('US'), ('USA')) t(s)
)
SELECT marisa_common_prefix(trie, 'USA', 10)
FROM rules;
```

The matching rules are U, US and USA; the limit caps the result count.

07 In practice

Build for a static set

Rebuild the aggregate when the dictionary changes. Normalize case and whitespace before both construction and querying if those differences should be ignored.

Full docs & examples ↗ Source ↗